ACCESCO LIVING PROJECT — TECHNICAL DATA ARCHITECTURE SPECIFICATION

Complete Data Requirements for Both Replenishment Engines

For the Accesco Living project, you need two separate data pipelines: one for warehouse/store replenishment and another for customer-level product depletion prediction. They can share the product catalogue, but they do not use the same prediction target or make the same decisions. The warehouse engine combines TimesFM 3.0 demand forecasting with deterministic inventory-planning rules; the customer engine uses XGBoost Survival AFT to estimate time until a customer is likely to run out of a product.
`Accesco_Dual_Replenishment_Engine_Model_Architecture_Updated.docx`

I'll break this down into the exact fields to collect, why each field is needed, where it should come from, and what to do if the data is currently unavailable.

1. Understand the three data priorities.

MUST
Required for the engine's core operation
Without these fields, the relevant calculation or prediction cannot be performed reliably.

SHOULD
Strongly recommended for accuracy
These fields improve reliability, help explain results, and prevent incorrect recommendations.

OPTIONAL
Enhancement for a later phase
Useful when enough data, operational processes, and verified labels are available.

One important distinction: input data is not the same as training data. For example, current inventory is essential to calculate a warehouse reorder recommendation, but historical SKU demand is needed to train or validate a demand-forecasting model. Similarly, a customer's latest purchase can generate a customer-product record, but reliable depletion outcomes are needed to train and evaluate the customer survival model.

PART A — WAREHOUSE/STORE-FACING REPLENISHMENT ENGINE

2. What this engine must calculate

For every SKU at every warehouse or dark store, the engine should determine:

- How much demand is expected over the upcoming forecast period.
- How much usable stock is currently available.
- How much stock is reserved or already committed.
- How much confirmed stock is on its way and when it will arrive.
- Whether stock is likely to run out before replenishment arrives.
- When replenishment should happen and how much to request.
- Whether the recommendation complies with supplier, budget, storage, and expiry constraints.

The engine needs both historical data for forecasting and current operational data for inventory planning.
`Accesco_Dual_Replenishment_Engine_Model_Architecture_Updated.docx`

3. Data category W1 — Master SKU/product catalogue

You already have a master SKU list. This is the starting point, but it needs to contain more than product names and IDs.

Field	Required?	Purpose
`sku_id`	**MUST**	Unique identifier for each sellable SKU.
`product_name`	**MUST**	Human-readable name for staff and reporting.
`category_id / category_name`	**SHOULD**	Helps group products and analyze demand patterns.
`brand`	**SHOULD**	Product identification and optional forecasting feature.
`unit_of_measure`	**MUST**	Defines whether quantities are in pieces, grams, kilograms, litres, packs, etc.
`unit_size`	**MUST**	Standard product quantity, such as 500 g, 1 kg, or 1 L.
`units_per_case / case_pack_size`	**MUST for case-based purchasing**	Converts required units into supplier case quantities.
`minimum_order_quantity`	**MUST if supplier imposes one**	Prevents invalid purchase recommendations.
`supplier_id / approved supplier mapping`	**MUST for purchase recommendations**	Identifies the supplier from whom the SKU can be ordered.
`purchase_price / unit cost`	**SHOULD**	Supports purchasing-budget checks.
`selling_price`	**SHOULD**	Supports business reporting and optional margin analysis.
`is_perishable`	**MUST**	Determines whether shelf-life and expiry rules apply.
`shelf_life_days`	**MUST for perishables when applicable**	Helps determine whether incoming stock can be sold before expiry.
`storage_requirement`	**SHOULD**	Identifies ambient, chilled, frozen, or other storage conditions.
`product_volume / storage_volume`	**SHOULD**	Supports storage-capacity calculations.
`product_weight`	**SHOULD**	Useful for storage, handling, and transport constraints.
`is_active`	**MUST**	Prevents replenishment of discontinued or inactive SKUs.
`valid_from, valid_to`	**SHOULD**	Tracks product catalogue changes over time.
`substitute_sku_id`	**OPTIONAL**	Supports substitution planning when a product is unavailable.

> **Important:** A 1 kg pack and a 500 g pack must not be treated as equivalent quantities. The model and planner need a consistent unit of measure. Otherwise, demand and inventory calculations can be wrong even if the SKU identifiers are correct.

4. Data category W2 — Warehouse, dark-store and location master

Every inventory and demand record must identify the node where it belongs.

Field	Required?	Purpose
`location_id`	**MUST**	Unique ID for each warehouse or dark store.
`location_name`	**MUST**	Identifies the node in reports and staff screens.
`location_type`	**MUST**	Distinguishes central warehouse, dark store, or other node types.
`location_status`	**MUST**	Prevents planning for inactive locations.
`parent_location_id`	**SHOULD**	Links a dark store to a supplying warehouse or parent node.
`service_area_id`	**SHOULD**	Helps associate demand with the location that serves it.
`operating_hours`	**SHOULD**	Helps schedule receiving and replenishment operations.
`storage_capacity`	**SHOULD**	Supports feasibility checks for incoming stock.
`storage_capacity_unit`	**MUST when capacity is used**	Specifies whether capacity is measured in volume, weight, pallet positions, etc.
`temperature/storage_zone`	**SHOULD**	Helps enforce product storage requirements.
`location_timezone`	**SHOULD**	Ensures daily demand and event timestamps are aligned correctly.

Required record granularity: demand and stock data must be linked to a `sku_id` and `location_id`. A company-wide total for an SKU is not sufficient to decide how much a particular dark store needs.

5. Data category W3 — Historical demand, sales and consumption

This is the main historical input for the TimesFM 3.0 forecasting component. The most important structure is a time series of demand for each SKU at each location. Transaction-level sales are useful, but the pipeline should also create a consistent daily or other approved time-bucket dataset.

Field	Required?	Purpose
`event_id / transaction ID`	**SHOULD**	Traces each source transaction and detects duplicates.
`event_timestamp`	**MUST**	Records when demand or a sale occurred.
`business_date`	**MUST in the aggregated dataset**	Assigns the event to the correct local trading day.
`sku_id`	**MUST**	Identifies the product.
`location_id`	**MUST**	Identifies where demand occurred.
`quantity_sold`	**MUST**	Measures fulfilled product demand.
`unit_of_measure`	**MUST**	Makes quantities comparable.
`order_id`	**SHOULD**	Links demand to an order.
`order_line_id`	**SHOULD**	Supports SKU-level order reconciliation.
`quantity_ordered`	**SHOULD**	Helps distinguish customer demand from fulfilled demand.
`quantity_fulfilled`	**MUST where available**	Measures the actual quantity supplied to customers.
`quantity_cancelled`	**SHOULD**	Helps identify demand lost due to cancellation.
`quantity_returned`	**SHOULD**	Prevents returns from being treated as positive demand.
`quantity_refunded`	**OPTIONAL**	Helps reconcile transactions and returns.
`sales_channel`	**SHOULD**	Separates app, web, or other channels if relevant.
`unit_selling_price`	**SHOULD**	Helps identify price-related demand changes.
`discount_amount / promotion indicator`	**SHOULD**	Helps explain promotional spikes.
`is_promotion`	**SHOULD**	Distinguishes normal demand from promotional demand.
`is_stockout_period`	**SHOULD**	Helps identify periods where recorded sales understate actual demand.
`data_source`	**SHOULD**	Identifies the system from which the record originated.

How this data should be transformed: The model should not simply be fed a list of unordered sales transactions. Create a standardized dataset with a record for each relevant `sku_id × location_id × date`.
Example (SKU-101 at Store-A): `1 Oct:` 18 units sold (Stockout: No) | `2 Oct:` 23 units sold (Stockout: No) | `3 Oct:` 0 units sold (Stockout: Yes) | `4 Oct:` 16 units sold (Stockout: No).
The zero on 3 October must not automatically be interpreted as zero customer demand. If the product was unavailable, sales may have been suppressed by the stockout. **Critical requirement:** preserve genuine zero-demand days, but distinguish them from missing records and periods when the product was unavailable. The history must be sufficiently long and consistent to support local backtesting; the exact minimum history and frequency should be confirmed after inspecting the data.

6. Data category W4 — Current inventory and inventory movements

The forecast tells you what demand may look like. Current inventory determines what action is necessary now.

Field	Required?	Purpose
`sku_id`	**MUST**	Identifies the stock item.
`location_id`	**MUST**	Identifies where the stock is held.
`snapshot_timestamp`	**MUST**	Indicates when the inventory quantity was recorded.
`on_hand_quantity`	**MUST**	Physical or recorded quantity currently held.
`reserved_quantity`	**MUST**	Quantity allocated to existing orders.
`damaged_quantity`	**SHOULD**	Identifies stock that cannot be sold.
`expired_quantity`	**MUST for perishables**	Excludes expired stock from usable inventory.
`quarantined_quantity`	**SHOULD**	Excludes stock held for inspection or other restrictions.
`blocked_quantity`	**SHOULD**	Excludes stock unavailable for sale or use.
`usable_quantity`	**MUST**	Quantity eligible to fulfil demand after exclusions.
`inventory_adjustment_quantity`	**SHOULD**	Records corrections, write-offs, and stock-count changes.
`movement_timestamp`	**MUST for movement history**	Orders stock movements chronologically.
`movement_type`	**MUST for movement history**	Distinguishes receipt, sale, transfer, damage, write-off, adjustment, etc.
`movement_quantity`	**MUST for movement history**	Records the quantity moved.
`source_location_id / destination_location_id`	**MUST for transfers**	Identifies the origin and destination of stock.
`lot_id / batch_id`	**MUST for lot-controlled perishables**	Enables expiry-aware planning.
`expiry_date`	**MUST for expiry-controlled products**	Prevents planning based on stock that will expire too soon.

> You should retain both current stock snapshots and historical inventory movements/snapshots. A current balance alone is enough to start some planning calculations, but historical stock records are useful for backtesting, stockout detection, and diagnosing inaccurate inventory data.

7. Data category W5 — Purchase orders, inbound supply and supplier lead times

These fields determine whether incoming stock will arrive in time to prevent a stockout.

Field	Required?	Purpose
`purchase_order_id`	**MUST for PO tracking**	Identifies the purchase order.
`supplier_id`	**MUST**	Identifies the supplier.
`sku_id`	**MUST**	Identifies the ordered product.
`source_location_id / receiving location_id`	**MUST**	Identifies where stock is coming from and where it will arrive.
`order_created_at`	**MUST for lead-time measurement**	Records when the PO was placed.
`quantity_ordered`	**MUST**	Records the quantity ordered.
`quantity_received`	**MUST when receipt records exist**	Records the quantity actually received.
`quantity_pending`	**MUST for inbound planning**	Records the remaining unreceived quantity.
`expected_arrival_timestamp`	**MUST for confirmed inbound planning**	Estimates when stock should arrive.
`actual_receipt_timestamp`	**MUST for historical lead-time calculation**	Records when stock actually arrived.
`purchase_order_status`	**MUST**	Distinguishes draft, confirmed, partially received, received, and cancelled orders.
`receipt_status`	**MUST when receipts are tracked**	Distinguishes pending, partial, complete, or rejected receipts.
`supplier_promised_lead_time_days`	**MUST if historical lead time is unavailable**	Supplies the initial lead-time assumption.
`actual_lead_time_days`	**MUST for historical lead-time analysis**	Measures actual time from the agreed PO event to receipt.
`supplier_on_time_history`	**SHOULD**	Helps assess supplier reliability when sufficient history exists.
`minimum_order_quantity`	**MUST where applicable**	Enforces supplier order limits.
`case_pack_size`	**MUST where applicable**	Rounds orders to permitted pack quantities.
`supplier_order_calendar`	**SHOULD**	Identifies ordering days, cut-off times, and holidays.
`purchase_price`	**SHOULD**	Supports budget and cost calculations.
`inbound_cancellation_reason`	**SHOULD**	Helps diagnose unreliable supply.

Important distinction: confirmed versus expected stock. Do not treat every open purchase order as guaranteed stock. An inbound quantity should be included according to its status, expected arrival date, and approved business rules. Cancelled orders must be excluded, and partial receipts must be split into received and pending quantities.

For a supplier with no historical PO/receipt data, use its stated lead time plus an Operations-approved buffer. Label that as a default assumption—not as an ML-predicted lead time.
`Accesco_Dual_Replenishment_Engine_Model_Architecture_Updated.pdf`

8. Data category W6 — Inter-location transfers

If the central warehouse supplies dark stores, transfer records are important even if the initial engine only recommends purchases.

Field	Required?	Purpose
`transfer_id`	**MUST when transfers are used**	Identifies a stock transfer.
`sku_id`	**MUST**	Identifies the product.
`source_location_id`	**MUST**	Identifies the supplying node.
`destination_location_id`	**MUST**	Identifies the receiving node.
`quantity_requested`	**MUST**	Records the requested transfer quantity.
`quantity_dispatched`	**MUST when dispatched**	Records the quantity sent.
`quantity_received`	**MUST when received**	Records the quantity accepted by the destination.
`transfer_created_at`	**MUST**	Records when the transfer was initiated.
`dispatch_timestamp`	**MUST when dispatched**	Records when stock left the source.
`expected_arrival_timestamp`	**SHOULD**	Supports expected inbound calculations.
`actual_arrival_timestamp`	**MUST for historical transfer lead time**	Measures transfer duration.
`transfer_status`	**MUST**	Distinguishes requested, approved, dispatched, received, and cancelled transfers.
`loss_or_damage_quantity`	**SHOULD**	Accounts for stock lost or damaged in transit.
`transfer_cost`	**OPTIONAL**	Supports later cost optimization.

> These fields help prevent double-counting stock: stock in transit should not be counted as usable on-hand stock at either location before the appropriate transfer event.

9. Data category W7 — Replenishment policy and business constraints

These are not necessarily model-training features. They are operational inputs to the deterministic inventory planner that turns the forecast into a feasible recommendation.

Field or rule	Required?	Purpose
`sku_id + location_id`	**MUST**	Defines which policy applies to which SKU and node.
`review_frequency`	**MUST**	Defines how often stock and recommendations are reviewed; daily monitoring is the current project assumption.
`planning_horizon_days`	**MUST**	Defines how far ahead demand is forecast and evaluated.
`supplier_lead_time_days`	**MUST**	Determines how long replenishment takes.
`safety_stock_quantity or approved safety-stock rule`	**MUST for policy-based planning**	Adds a buffer against uncertainty.
`reorder_point or rule for calculating it`	**MUST**	Defines when stock should trigger replenishment.
`target_stock_level / order-up-to level`	**MUST**	Defines the desired stock level after replenishment.
`minimum_order_quantity`	**MUST where applicable**	Enforces supplier constraints.
`case_pack_size`	**MUST where applicable**	Rounds recommendations to valid quantities.
`shelf_life_days / minimum remaining shelf life`	**MUST for applicable perishables**	Avoids ordering stock likely to expire before sale.
`storage_capacity and current capacity usage`	**SHOULD**	Prevents infeasible recommendations.
`purchasingbudget/budgett limit`	**SHOULD**	Flags recommendations exceeding budget.
`supplier order days and cut-off times`	**SHOULD**	Helps calculate feasible order timing.
`service-level target`	**SHOULD**	Supports an approved safety-stock policy.
`priority thresholds`	**MUST**	Defines how stockout risk is translated into operational priority.
`transfer-versus-purchase policy`	**OPTIONAL for initial purchase-only scope**	Supports later sourcing decisions between transfers and new purchases.
`policy_effective_from / policy_version`	**SHOULD**	Allows decisions to be reproduced and audited.

> **Do not invent these values.** Where Accesco has not approved safety-stock rules, service levels, or reorder policies, keep the fields marked as pending and agree the values with Operations. A model should not silently turn an unapproved assumption into a business rule.

10. Data category W8 — Calendar and demand drivers

These fields are useful for explaining demand fluctuations, especially around promotions and recurring events.

Field	Priority	Purpose
`business_date`	**MUST**	Aligns daily demand records.
`day_of_week`	**DERIVED**	Captures recurring weekly demand patterns.
`week_of_year / month`	**DERIVED**	Captures seasonal patterns.
`public_holiday_flag`	**SHOULD**	Identifies holiday-related demand changes.
`promotion_id / promotion flag`	**SHOULD**	Identifies promotional demand.
`promotion_start / promotion_end`	**SHOULD**	Identifies when the promotion was active.
`price_change`	**SHOULD**	Helps explain demand changes caused by pricing.
`store_open_flag`	**MUST if operating days vary**	Distinguishes closed days from zero-demand days.
`product_available_flag`	**SHOULD**	Helps distinguish observed low demand from constrained sales.
`Weather or external events`	**OPTIONAL**	Potential future demand drivers if relevant and reliable.

> The first release does not need every possible external feature. Start with reliable internal sales, stock, availability, and calendar data, then test whether extra variables improve out-of-sample forecasts.

11. Data category W9 — Staff approval and recommendation outcomes

Because the initial warehouse engine must remain reviewable by Operations, you also need to capture what happened after a recommendation was generated.

Field	Required?	Purpose
`recommendation_id`	**MUST**	Uniquely identifies each recommendation.
`generated_at`	**MUST**	Records when it was produced.
`sku_id + location_id`	**MUST**	Identifies the affected product and node.
`forecast_horizon`	**MUST**	Records the period the forecast covers.
`forecast_units`	**MUST**	Records predicted demand.
`inventory_position`	**MUST**	Records the stock position used by the planner.
`suggested_quantity`	**MUST**	Records the proposed replenishment amount.
`suggested_order_date`	**MUST**	Records the recommended action date.
`stockout_risk_band`	**MUST**	Communicates urgency using an approved rule.
`expiry_or_capacity_warning`	**MUST where relevant**	Explains infeasible or risky recommendations.
`model_version and policy_version`	**MUST**	Makes the recommendation reproducible.
`review_status`	**MUST**	Records pending, approved, rejected, or modified status.
`reviewer_id`	**MUST**	Identifies the authorized reviewer.
`approved_quantity`	**MUST when approved**	Records the final authorized quantity.
`override_reason`	**MUST when changed or rejected**	Explains why staff changed the recommendation.
`actual_order_id / transfer_id`	**SHOULD**	Links the decision to the actual action.
`actual_outcome`	**SHOULD**	Helps evaluate whether the action prevented stockout or created excess stock.

> This feedback is essential for monitoring and process improvement. However, staff approval does not automatically make an outcome a correct model-training label. You must separately define the outcomes used to assess forecast accuracy and replenishment performance.

PART B — CUSTOMER-FACING REPLENISHMENT ENGINE

12. What this engine must calculate

For each eligible customer-product pair, the engine should estimate:

- The likely time until the product is depleted.
- The estimated depletion date or a clearly defined uncertainty range.
- How the estimate changes with pack size, purchase quantity, and past purchase intervals.
- Whether enough evidence exists to personalize the estimate.
- Whether a reminder is permitted and appropriate.
- Whether the customer has already reported a purchase, delayed consumption, or still has product remaining.

The selected model is XGBoost Survival AFT. It must handle cases where the event has not been observed by the end of the data window. When actual depletion cannot be observed, time until the next qualifying purchase can be used as a proxy, but it must not be presented as a directly observed depletion event.
`Accesco_Dual_Replenishment_Engine_Model_Architecture_Updated.pdf`

13. Data category C1 — Customer identity and account mapping

You need a reliable way to link purchases to the same customer over time.

Field	Required?	Purpose
`customer_id`	**MUST**	Stable pseudonymous identifier linking customer activity.
`account_id`	**SHOULD**	Links the customer to an authenticated app account.
`household_id`	**OPTIONAL/useful**	Allows a household to share purchases and consumption information when explicitly supported.
`profile_id`	**OPTIONAL**	Supports multiple household members or profiles.
`account_created_at`	**SHOULD**	Helps establish the beginning of the observable customer history.
`account_status`	**MUST for live reminders**	Prevents sending reminders to inactive or deleted accounts.
`identity_link_status`	**MUST**	Indicates whether a purchase can reliably be attributed to the customer.
`customer_timezone`	**SHOULD**	Helps schedule reminders at appropriate local times.
`preferred_language`	**SHOULD**	Supports understandable notifications.
`notification_preferences`	**MUST for reminders**	Records allowed reminder channels and user choices.

> Use a pseudonymous customer ID in the model dataset. Do not feed names, phone numbers, email addresses, or full delivery addresses into the prediction model unless there is a separately justified and approved need. Contact details should be kept in the notification/account system.

> **Current project gap:** customer accounts and linked order history have not been confirmed as available. If purchases cannot be tied to a stable customer ID, personalized training and reminders must wait until the identity and event-capture process is implemented.

14. Data category C2 — Customer purchase transactions

This is the most important source dataset for the customer engine.

Field	Required?	Purpose
`purchase_event_id`	**MUST**	Unique identifier for the purchase event.
`customer_id`	**MUST for personalization**	Links the event to a customer.
`order_id / receipt_id`	**MUST where available**	Identifies the transaction and helps deduplicate records.
`purchase_timestamp`	**MUST**	Records when the purchase happened.
`product_id / sku_id`	**MUST**	Identifies the purchased product.
`quantity_purchased`	**MUST**	Records the amount purchased.
`unit_of_measure`	**MUST**	Defines whether the quantity is pieces, grams, litres, etc.
`pack_size`	**MUST**	Defines the amount in each purchased pack.
`normalized_quantity`	**MUST**	Converts different package sizes into a common product unit.
`purchase_price`	**SHOULD**	Records the actual price paid.
`discount_amount`	**SHOULD**	Helps identify promotions and unusual buying behaviour.
`purchase_channel`	**MUST**	Distinguishes Accesco app/web purchases from external purchases, if collected.
`purchase_status`	**MUST**	Excludes cancelled or invalid transactions.
`return_quantity`	**MUST where returns occur**	Adjusts the effective purchased quantity.
`substitution_flag`	**SHOULD**	Identifies whether the purchased item replaced another product.
`source_event_id / data_source`	**SHOULD**	Records where the event came from.
`recorded_at`	**SHOULD**	Distinguishes the purchase time from the time it entered the system.

Example of why quantity normalization matters: Suppose the same customer buys cooking oil in two different pack sizes:
• **Purchase date: 1 September** | Product: Oil SKU-10 | Pack size: 1 L | Quantity purchased: 1 | Total amount: **1 L**
• **Purchase date: 22 September** | Product: Oil SKU-10 | Pack size: 500 mL | Quantity purchased: 2 | Total amount: **1 L**
Both purchases represent 1 litre of oil. The model must not interpret the second purchase as a larger purchase merely because the pack count differs.

Your earlier proposed fields—product ID, product name, purchase date, size, price, and time since the previous purchase—are a useful start. To make them usable for training, add a stable customer ID, normalized quantity, transaction validity, source channel, and a defined outcome/label.

15. Data category C3 — Previous purchase and repeat-purchase history

These fields are usually derived from the transaction table. They should be calculated by the pipeline rather than manually maintained.

Field	Required?	Purpose
`previous_purchase_timestamp`	**MUST where a previous purchase exists**	Identifies the previous qualifying purchase.
`days_since_previous_purchase`	**MUST where available**	Measures the interval since the previous purchase.
`previous_purchase_quantity`	**SHOULD**	Helps compare changes in purchased quantity.
`previous_pack_size`	**SHOULD**	Helps account for different package sizes.
`previous_purchase_price`	**SHOULD**	Helps identify changed purchase conditions.
`purchase_count_for_customer_sku`	**MUST**	Measures how many qualifying purchases are observed.
`first_purchase_flag`	**MUST**	Identifies a first observed purchase for that customer-product pair.
`average_purchase_interval_days`	**SHOULD**	Summarizes prior intervals when sufficient history exists.
`median_purchase_interval_days`	**SHOULD**	Provides a robust interval estimate less affected by outliers.
`minimum_purchase_interval_days`	**OPTIONAL**	Helps identify unusual short intervals.
`maximum_purchase_interval_days`	**OPTIONAL**	Helps identify unusually long intervals.
`purchase_interval_variability`	**SHOULD**	Indicates whether the customer's pattern is consistent.
`days_since_latest_purchase`	**MUST**	Measures elapsed time since the latest qualifying purchase.
`quantity_trend`	**OPTIONAL**	Identifies whether purchase quantity is increasing or decreasing.
`product_substitution_history`	**SHOULD**	Helps avoid treating a substitute purchase as a completely unrelated event.

How to handle a first-time customer-product purchase:
If the customer has never bought the SKU through the tracked system:

- Set `first_purchase_flag = 1`.
- Set `purchase_count_for_customer_sku = 1`.
- Set `previous_purchase_timestamp` and `days_since_previous_purchase` to null, not zero.
- Do not invent an average or median personal purchase interval.
- Use a clearly defined cold-start estimate only if suitable product-level or category-level data is available.
- Label the result as a population-based estimate, not a personalized prediction.

A customer who has one observed purchase is not necessarily a person who has never bought the product before. They may have purchased it elsewhere. The system should therefore distinguish first observed Accesco purchase from confirmed first-ever purchase, because these are different facts.

16. Data category C4 — Product consumption and depletion information

This category provides the evidence needed to estimate when the customer may run out.

Field	Required?	Purpose
`sku_id / normalized product ID`	**MUST**	Identifies the product being tracked.
`normalized_quantity_purchased`	**MUST**	Provides the quantity available for consumption.
`purchase_timestamp`	**MUST**	Establishes the start of the observation period.
`depletion_observed_flag`	**MUST for a direct-depletion label**	Indicates whether actual depletion was observed.
`depletion_timestamp`	**MUST if depletion is directly observed**	Provides the actual depletion event time.
`depletion_source`	**MUST**	Distinguishes user confirmation, measured usage, or a proxy.
`remaining_quantity`	**SHOULD when collected**	Indicates whether product remains at a given time.
`consumption_rate`	**SHOULD if reliable measurements exist**	Estimates usage per day or week.
`household_size`	**OPTIONAL / potentially useful**	May help explain household consumption, with user permission.
`usage_frequency`	**OPTIONAL/useful**	Captures how often a product is used.
`product_usage_context`	**OPTIONAL**	Provides product-specific context, if collected and justified.
`depletion_feedback_timestamp`	**MUST when feedback is recorded**	Records when the customer reported depletion.
`depletion_feedback_method`	**MUST when feedback is recorded**	Records whether depletion was reported in-app, via a prompt, or through another method.
`feedback_confidence`	**SHOULD**	Distinguishes explicit confirmation from uncertain feedback.

Direct observation versus a proxy — There are three different levels of evidence:

1. **Directly observed depletion:** The customer confirms that the product has run out, or a reliable measurement establishes depletion.
2. **Observed repeat-purchase interval:** The customer buys the same product again. This shows a repeat-purchase event, but does not prove that the previous pack was empty.
3. **Estimated consumption:** A model estimates how long the purchased quantity might last based on available product and customer information.

Keep these labels separate in the dataset. A purchase on a particular date must not automatically be treated as the exact depletion date of the previous purchase.

17. Data category C5 — Survival-model target and censoring fields

This is a critical data category because XGBoost Survival AFT is a time-to-event model, not a standard classifier that simply predicts “reorder” or “do not reorder.” Each training example needs a well-defined start time, event definition, observed duration, and censoring status.

Field	Required?	Purpose
`customer_id`	**MUST**	Identifies the customer in the training example.
`sku_id`	**MUST**	Identifies the product.
`observation_start_timestamp`	**MUST**	Defines when the time-to-event clock starts.
`observation_end_timestamp`	**MUST**	Defines the end of the observed period.
`duration_days`	**MUST**	Measures elapsed time from the start to the observed event or censoring point.
`event_observed_flag`	**MUST**	Indicates whether the defined event occurred during observation.
`event_type`	**MUST**	Distinguishes confirmed depletion from a repeat-purchase proxy or another event definition.
`censoring_flag`	**MUST**	Identifies cases where the event was not observed before the data cutoff or observation ended.
`data_cutoff_timestamp`	**MUST**	Makes the observation window reproducible.
`label_lower_bound / label_upper_bound`	**MUST for interval-censored labels**	Represents the time interval within which the event is known to have occurred, if applicable.
`label_definition_version`	**MUST**	Identifies the approved target definition.
`qualifying_purchase_rule`	**MUST**	Defines what counts as a repeat purchase for the target.
`return_or_cancellation_rule`	**MUST**	Defines how invalid or reversed transactions are handled.
`substitution_rule`	**MUST**	Defines whether a substitute product qualifies as a repeat event.

Example of a censored observation: Suppose a customer buys a 1 kg product on 1 October. The data collection period ends on 28 October, and no depletion or qualifying repeat purchase has been observed. The correct training record is conceptually:

- Observation start: 1 October
- Observation end: 28 October
- Observed duration: 27 days, subject to the project’s timestamp convention
- Event observed: No
- Censored: Yes

It would be incorrect to claim that the product was depleted on 28 October merely because the dataset ended then. For XGBoost Survival AFT, the data-preparation pipeline must convert these records into the supported survival-label representation. The team must also define the event and censoring rules before training; the model cannot repair an incorrect target definition. `Accesco_Dual_Replenishment_Engine_Model_Architecture_Updated.pdf`

18. Data category C6 — Offline purchases and purchases from other shops

You previously wanted the engine to account for a customer buying the same product outside Accesco. This requires an additional data-capture mechanism, because Accesco cannot infer every external purchase from its own order history. A practical initial approach is to allow customers to report an external purchase or update their remaining quantity.

Field	Required?	Purpose
`customer_id`	**MUST for personalized tracking**	Links the external purchase to the customer.
`sku_id / product mapping`	**MUST**	Identifies the equivalent Accesco product.
`external_purchase_timestamp`	**MUST**	Records when the outside purchase happened.
`external_quantity`	**MUST**	Records the quantity purchased.
`external_unit_of_measure`	**MUST**	Defines the quantity unit.
`external_pack_size`	**MUST**	Enables quantity normalization.
`external_store_name`	**OPTIONAL**	Identifies the outside retailer if the customer provides it.
`external_purchase_price`	**OPTIONAL**	Records the reported price if useful.
`purchase_evidence_type`	**MUST**	Distinguishes manual entry, receipt scan, barcode scan, or another method.
`receipt_or_scan_reference`	**OPTIONAL**	Supports verification if receipt capture is implemented.
`product_match_confidence`	**MUST if automated matching is used**	Indicates whether the external product matches an Accesco SKU reliably.
`customer_confirmation_flag`	**SHOULD**	Indicates whether the customer confirmed the mapped product.
`recorded_at`	**MUST**	Records when the external purchase was added to the system.

How this information should affect the engine:

- If a customer reports buying the same product elsewhere, record that event and update the latest known purchase date and quantity.
- If a receipt is scanned, match the product to the master catalogue using a barcode or verified product mapping.
- If the match is uncertain, request confirmation or leave it unmatched; do not silently assign it to a similar SKU.
- If the customer reports that the product still has stock remaining, use that feedback to update the observation state.
- If no external purchase is reported, do not assume that the customer bought nothing elsewhere.

External purchase reporting is a useful enhancement, but it will not provide complete coverage unless customers consistently use it. Predictions should communicate that uncertainty.

19. Data category C7 — Product catalogue for customer predictions

The warehouse and customer engines can share a governed product catalogue, but the customer engine also needs consistent product matching and pack-size information.

Field	Required?	Purpose
`sku_id`	**MUST**	Identifies the sellable SKU.
`product_name`	**MUST**	Displays the product in the app.
`canonical_product_id`	**SHOULD**	Groups different SKUs or pack sizes of the same underlying product, when appropriate.
`category_id`	**SHOULD**	Enables category-level cold-start estimates.
`brand`	**SHOULD**	Helps distinguish similar products.
`unit_size`	**MUST**	Identifies the pack size.
`unit_of_measure`	**MUST**	Standardizes quantity calculations.
`barcode / GTIN`	**SHOULD**	Supports barcode and receipt matching.
`equivalent_product_mapping`	**SHOULD**	Maps external products and equivalent pack sizes to a canonical product.
`substitute_product_mapping`	**SHOULD**	Distinguishes equivalents from merely similar substitutes.
`product_status`	**MUST**	Excludes discontinued or unavailable catalogue items from inappropriate suggestions.
`shelf_life_days`	**OPTIONAL for the core personal estimate**	May support product-specific depletion assumptions, but shelf life is not the same as actual usage rate.

> Do not merge products just because their names are similar. A 500 g product, a 1 kg product, and a different formulation may require separate SKU records even when they share a canonical product grouping.

20. Data category C8 — Cold-start and population-level data

When a customer has little or no personal purchase history, the engine needs a fallback rather than pretending to know that customer's habits.

Field or dataset	Required?	Purpose
`canonical_product_id / category_id`	**MUST for grouped estimates**	Defines the product group used for fallback estimates.
`normalized_quantity`	**MUST**	Makes different pack sizes comparable.
`purchase_timestamp`	**MUST for interval proxies**	Establishes observed purchase timing.
`customer_id`	**MUST in historical event data**	Supports correct grouping by customer.
`event_type`	**MUST**	Keeps observed depletion and repeat-purchase proxies separate.
`duration_days`	**MUST for eligible survival examples**	Supplies observed time-to-event or censored duration.
`event_observed_flag`	**MUST**	Distinguishes observed from censored events.
`customer_count / sample-size measure`	**SHOULD**	Helps establish whether a fallback estimate is based on sufficient observations.
`population_interval_distribution`	**SHOULD**	Summarizes variation in observed intervals for comparable products.
`estimate_source`	**MUST**	Identifies whether an estimate is population-based or personalized.
`estimate_confidence / uncertainty representation`	**SHOULD**	Communicates the reliability of the estimate.

> Your earlier project direction was to start with population-level information and gradually rely more on individual history after roughly 3–5 observed purchases for a customer-product pair. Treat that as an initial design threshold to validate —not a universal statistical guarantee. A purchase count alone does not establish that the labels are correct or that the model is calibrated.

21. Data category C9 — Customer consent, reminder eligibility and notification preferences

The prediction engine and notification system should be separate. A predicted depletion date does not automatically authorize a notification.

Field	Required?	Purpose
`customer_id`	**MUST**	Links notification eligibility to the account.
`consent_status`	**MUST**	Records whether the relevant processing or communication is permitted.
`consent_timestamp`	**MUST when consent is recorded**	Establishes when consent was granted or changed.
`consent_source`	**SHOULD**	Records where consent was collected.
`consent_policy_version`	**SHOULD**	Identifies the consent wording or policy applicable at the time.
`notification_opt_in`	**MUST**	Indicates whether the customer permits the relevant reminders.
`allowed_notification_channels`	**MUST**	Defines whether push notifications, WhatsApp, or other approved channels may be used.
`quiet_hours`	**SHOULD**	Prevents reminders during disallowed times.
`preferred_language`	**SHOULD**	Supports appropriate notification content.
`notification_frequency_limit`	**MUST**	Prevents excessive reminders.
`last_notification_timestamp`	**MUST**	Supports frequency and duplicate-send controls.
`last_notification_status`	**MUST**	Records queued, sent, delivered, failed, or other supported status.
`unsubscribe_or_opt_out_timestamp`	**MUST when applicable**	Prevents further messages after opt-out.
`reminder_feedback`	**SHOULD**	Records whether the customer found the reminder useful or inaccurate.

> Keep the minimum required personal information in the prediction pipeline. Consent, retention, deletion, and channel-specific requirements must be reviewed before production reminders are enabled.

22. Data category C10 — Shared inventory and product availability

Customer reminders should also consider whether the product is actually available for purchase. A reminder for an unavailable item can create a poor customer experience.

Required & Recommended Fields for Category C10

- `sku_id` (**MUST**) — Identifies the product.
- `location_id / service area` (**MUST for local availability**) — Identifies the relevant fulfilment area.
- `availability_status` (**MUST**) — Indicates whether the product is available to order.
- `available_quantity` (**SHOULD**) — Provides a more informative availability signal.
- `availability_timestamp` (**MUST**) — Identifies when availability was checked.
- `customer_service_area_id` (**MUST if availability varies geographically**) — Links the customer to the relevant fulfilment area.
- `expected_restock_date` (**OPTIONAL**) — Supports a later restock-aware reminder flow.

If the product is unavailable, the notification service can delay the reminder, suppress it, or offer an approved alternative. That decision should be governed by explicit business rules.

23. Data category C11 — Reminder outcomes and customer feedback

This dataset allows you to evaluate whether reminders are useful and whether the predictions are correct.

Field	Required?	Purpose
`reminder_id`	**MUST**	Identifies a reminder.
`customer_id + sku_id`	**MUST**	Identifies the customer-product pair.
`prediction_id`	**MUST**	Links the reminder to the prediction that triggered it.
`predicted_depletion_date`	**MUST**	Records the predicted date used by the reminder service.
`prediction_generated_at`	**MUST**	Records when the estimate was created.
`prediction_model_version`	**MUST**	Makes results traceable.
`reminder_eligibility_reason`	**MUST**	Records why the reminder was allowed or suppressed.
`reminder_sent_at`	**MUST if sent**	Records when the reminder was sent.
`delivery_status`	**MUST**	Records whether the notification was successfully processed or delivered.
`customer_feedback_timestamp`	**SHOULD**	Records when feedback was provided.
`customer_reported_depletion_date`	**SHOULD**	Provides an observed outcome if the customer supplies it.
`customer_reported_remaining_quantity`	**SHOULD**	Helps identify when the depletion estimate was too early or late.
`next_qualifying_purchase_timestamp`	**MUST when a repeat purchase is observed**	Provides the repeat-purchase event used for proxy evaluation.
`reminder_action`	**SHOULD**	Records whether the customer viewed, dismissed, or acted on the reminder, where permitted and available.
`opt_out_after_reminder`	**MUST when applicable**	Helps enforce user controls and monitor notification fatigue.

> A customer purchasing the product after a reminder does not, by itself, prove that the reminder was accurate. Evaluate depletion prediction separately from notification engagement.

PART C — DATA THAT BOTH ENGINES SHARE

24. Common data requirements

These datasets should have one governed definition so that the two engines do not disagree about product IDs, quantities, or location identities.

Shared dataset	Key fields
`Product master`	SKU ID, product name, category, pack size, unit of measure, active status
`Product mappings`	Canonical product ID, barcode, equivalent products, substitute mappings
`Location master`	Location ID, type, status, service area, parent location
`Calendar`	Timestamp, business date, timezone, holiday and operating-day indicators
`Source metadata`	Source system, record ID, ingestion timestamp, update timestamp
`Data-quality results`	Missing-field flags, duplicate flags, invalid quantities, mapping confidence
`Model and pipeline metadata`	Dataset version, feature version, model version, run timestamp
`Access and audit metadata`	Data owner, access permissions, retention rule, audit history

> For integration, the relevant OMS, inventory-management, WMS, and TMS source systems need to be checked to establish which system owns each field and how frequently it is updated. Their schemas and event contracts have not all been confirmed. `Accesco_Dual_Replenishment_Engine_Model_Architecture_Updated.docx`

PART D — WHAT DATA SHOULD ACTUALLY BE FED INTO EACH MODEL?

The full database is not fed directly into a model. The pipeline cleans the source data, calculates derived features, creates training labels, and selects the appropriate fields.

25. Warehouse engine — model and planner inputs

A. TimesFM 3.0 forecasting input

One time series per SKU and warehouse/store node.

- `sku_id`
- `location_id`
- `business_date` or approved time-bucket timestamp
- Historical `quantity_sold` or a consistently defined demand measure
- Correct handling of missing days, genuine zero-demand days, and stockout-constrained periods
- Optional validated calendar and promotion covariates, if supported by the selected interface and shown to help

B. Deterministic inventory planner input

- Forecast demand over the planning and lead-time horizon
- Current usable inventory
- Reserved or committed inventory
- Confirmed inbound quantities and expected arrival dates
- Supplier lead time and approved buffer
- Safety-stock and reorder-point policy
- Target stock level
- Minimum order quantity and case-pack size
- Shelf-life, expiry, and storage-capacity constraints
- Budget and supplier-ordering constraints, where applicable

The planner then produces a recommendation; the forecast alone does not determine an order quantity. The approved design explicitly calls for inventory-position calculations and constraint checks.
`Accesco_Dual_Replenishment_Engine_Model_Architecture_Updated.pdf`

26. Customer engine — model inputs

A. Customer-product feature input

One prediction record per eligible customer-product pair.

- Stable pseudonymous `customer_id`
- `sku_id` and, where useful, canonical product ID/category
- Normalized purchased quantity and pack size
- Purchase timestamp and days since latest purchase
- First-observed-purchase flag
- Number of qualifying observed purchases
- Previous purchase interval and historical interval summaries, when available
- Product-level or population-level features for cold-start cases
- Verified feedback and consumption information, when available and suitable

B. Survival-training label

- Observation start timestamp
- Observation end or data-cutoff timestamp
- Duration until the defined event or censoring
- Event-observed flag
- Censoring status
- Event type: observed depletion or explicitly labelled repeat-purchase proxy
- Approved rules for returns, cancellations, quantity changes, and substitutions

C. Separate reminder-eligibility input

- Customer consent and notification preferences
- Last reminder and frequency-limit status
- Current product availability
- Prediction date and uncertainty
- Customer feedback and suppression rules

Consent and notification eligibility should be checked by the reminder service, not treated as variables that make a depletion prediction more accurate.

PART E — THE OUTPUT DATA YOU MUST STORE

The engines should store their predictions and decisions, not just return a number to the app or dashboard.

27. Warehouse engine output (At minimum, store)

- SKU and location identifiers
- Forecast horizon and forecast demand
- Inventory position and usable stock
- Confirmed inbound stock
- Estimated demand during supplier lead time
- Reorder threshold and target stock level
- Suggested replenishment quantity and recommended timing
- Stockout-risk band
- Expiry, capacity, or budget warnings
- Model version, policy version, and generation timestamp
- Staff review status, approved quantity, and override reason
- Actual purchase/transfer reference and subsequent outcome, when available

28. Customer engine output (At minimum, store)

- Customer and product identifiers
- Predicted time to depletion
- Estimated depletion date
- Uncertainty or confidence representation
- Whether the estimate is population-based or personalized
- Whether the target is based on directly observed depletion or a purchase-interval proxy
- Model version and prediction timestamp
- Reminder eligibility result and reason
- Reminder channel and delivery status, if sent
- Subsequent customer feedback or qualifying purchase event

> Do not present a precise depletion date as certain when the model has only weak evidence. The user interface should communicate that it is an estimate.